

ASSIGNMENT 1 — MODEL RESEARCH, BENCHMARKING & RECOMMENDATION

Comparative Study of Speech-to-Text Models

for Noisy, Multi-Accent Real-World Audio

Prepared by: Kunal Chachane

Date: 26 September 2026

Version 1.0

Table of Contents

1. Introduction & Problem Statement.....	3
2. Research: Candidate Model Landscape.....	4
2.1 OpenAI Whisper (base / small)	4
2.2 Faster-Whisper (CTranslate2)	5
2.3 Wav2Vec2	5
2.4 Model Landscape Summary	6
3. Benchmark Methodology	7
4. Comparative Results & Analysis.....	8
5. Recommendation.....	10
6. Conclusion.....	13

1. Introduction & Problem Statement

Ask even the most experienced call agents, and they will be quick to recite problems that stemmed from real-world specifics of phone call processing. Background noises of an open-plan office, a passing car, poor compression, overlapping speech, and multitudes of accents from all corners of the world and various fluency levels are just the daily hurdles that calling centers have to overcome on a daily basis. This is just a working environment that a virtual assistant would have to adapt to – not the preferred scenario.

A speech-to-text assistant will only be as good as its underlying transcription, with all the inaccuracies inevitably propagated to intent identification, knowledge retrieval, and final answer returned by the called agent.

Therefore, I consider it wise to invest time and effort into researching and preparing the best possible option for the lowest possible risk.

The report adheres exactly to the requirements given in the assignment brief: 5 potential model families were researched for consideration, further narrowed to the 3 most viable options according to preliminary research; a detailed benchmark of the shortlisted group was conducted, and the results evaluated with specific criteria in mind to provide a final recommendation, along with a suggested optimization plan, fine-tuning approach, and implementation considerations.

2. Research: Candidate Model Landscape

For this project, three ASR models were selected for detailed evaluation: **Whisper Small, Faster-Whisper Small, and Wav2Vec2 Base 960h**. The selection provides a comparison between a general-purpose Transformer-based ASR model, an optimized implementation of the same Whisper model family, and a different CTC-based speech-recognition approach.

2.1 Whisper Small

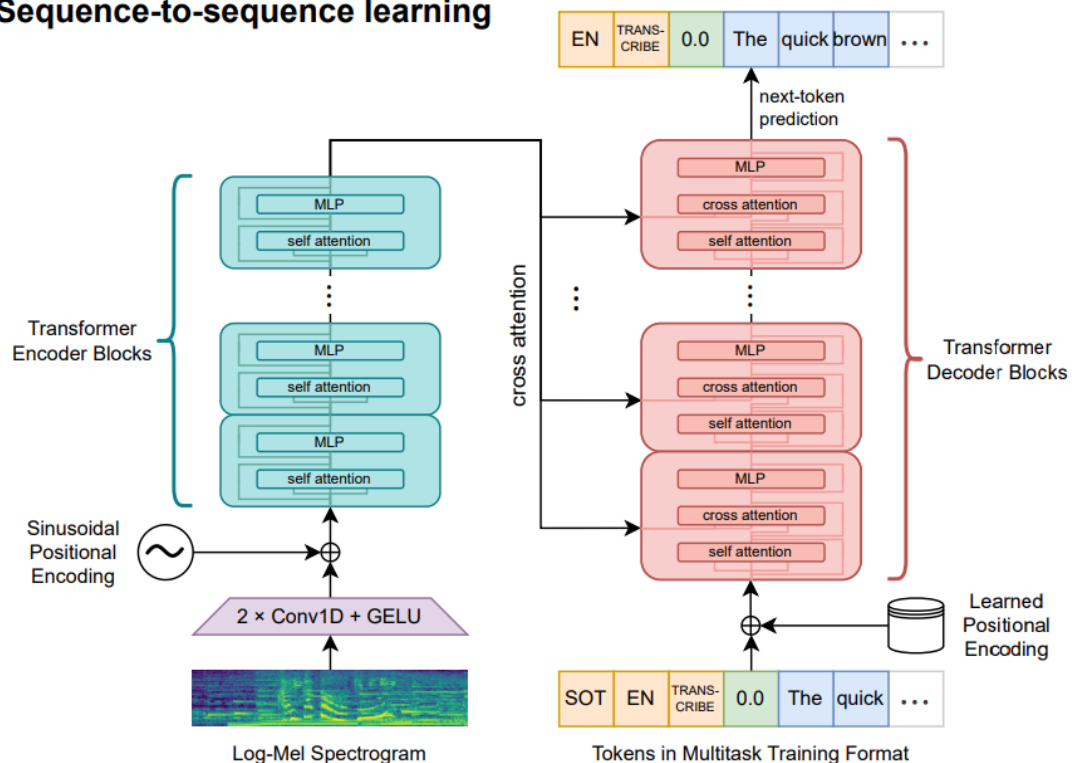
Whisper Small is an encoder-decoder Transformer model designed for automatic speech recognition. The input audio is converted into a log-Mel spectrogram and processed by the encoder, while the decoder generates the transcription.

The Whisper model family was trained using approximately **680,000 hours of multilingual and multitask audio collected from the web**. The Small checkpoint contains approximately **244 million parameters** and is released under the **MIT license**.

The main limitation is computational cost. Compared with smaller or highly optimized ASR systems, Whisper can require more processing time and memory, particularly when running on a CPU.

Role in this benchmark: Whisper Small is used as the general-purpose baseline.

Sequence-to-sequence learning



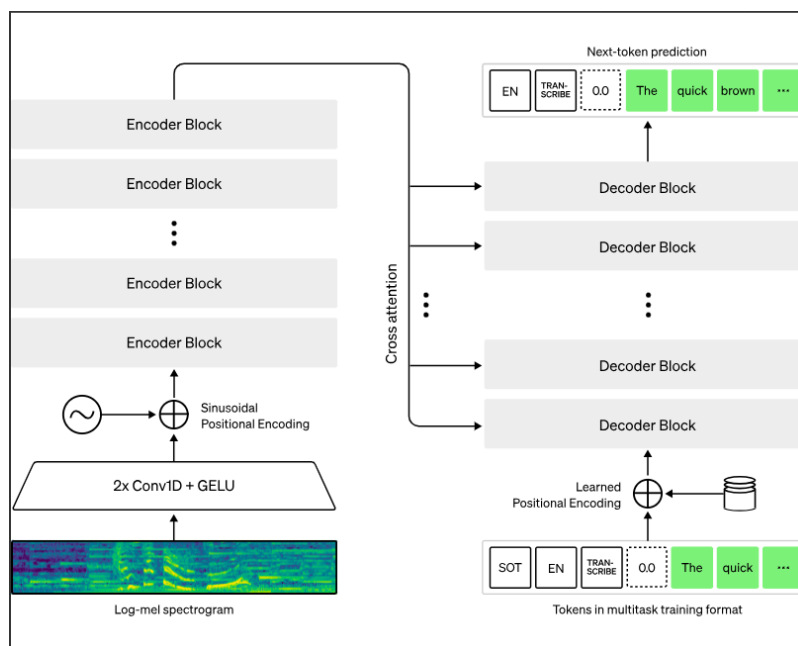
2.2 Faster-Whisper Small

Faster-Whisper is an optimized implementation of Whisper that uses the **CTranslate2** inference engine. It uses Whisper model weights rather than introducing a separately trained ASR model.

Faster-Whisper is released under the **MIT license**. Since it uses the Whisper model family, it retains the underlying capabilities of the selected Whisper checkpoint while providing a different inference implementation.

Its main advantage for this project is deployment efficiency. The main consideration is that performance still depends on the available CPU/GPU hardware and the selected inference configuration.

Role in this benchmark: Faster-Whisper Small is used to evaluate whether optimized Whisper inference provides practical improvements in speed and resource consumption.



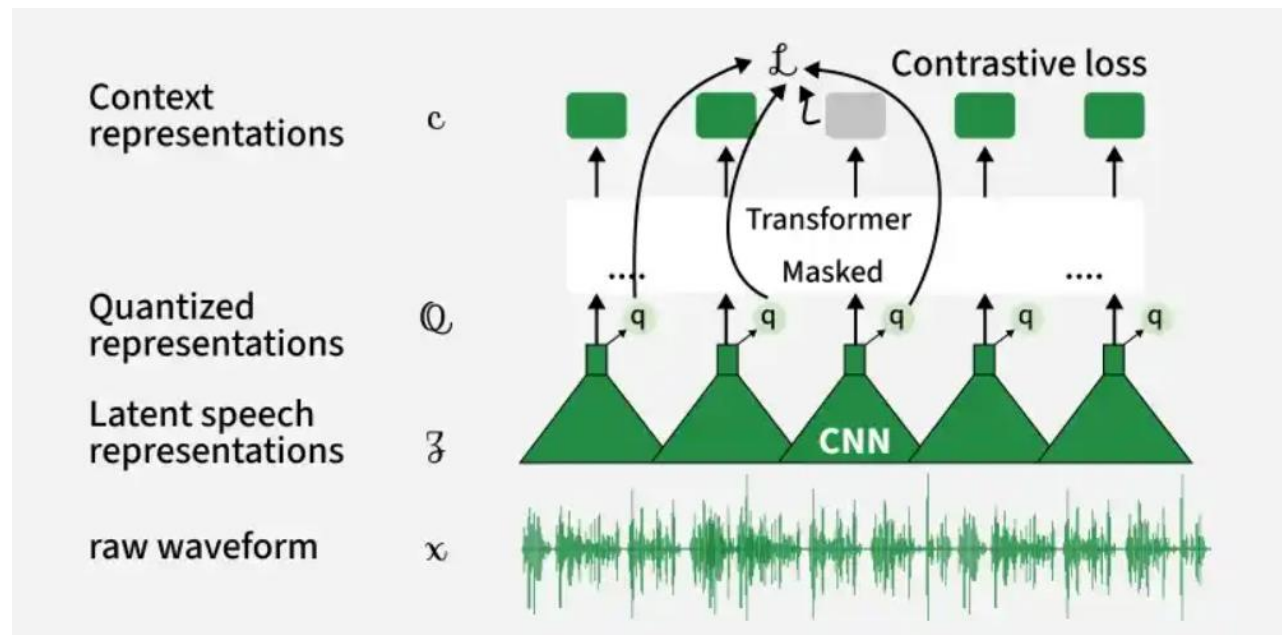
2.3 Wav2Vec2 Base 960h

Wav2Vec2 Base 960h follows a different approach from Whisper. It uses self-supervised learning to learn speech representations from audio and is then fine-tuned for speech recognition. The selected checkpoint uses a **CTC (Connectionist Temporal Classification)** objective for generating transcriptions.

The exact checkpoint used in this project is **facebook/wav2vec2-base-960h**. It was pretrained and fine-tuned using **960 hours of LibriSpeech audio** and is intended for English speech recognition. The model is released under the **Apache-2.0 license**.

Wav2Vec2 provides a useful comparison because its architecture and training approach differ from Whisper. It is also well supported through the Hugging Face Transformers ecosystem.

Role in this benchmark: Wav2Vec2 Base 960h is used as the alternative ASR architecture for comparison with the two Whisper-based systems.



2.4 Why These Three Models Were Selected

The three models were selected because they allow the project to examine both **model performance and deployment efficiency**.

Model	Architecture	Purpose
Whisper Small	Encoder-decoder Transformer	General-purpose baseline
Faster-Whisper Small	Whisper + CTranslate2	Inference efficiency
Wav2Vec2 Base 960h	Transformer + CTC	Alternative English ASR approach

Whisper Small and Faster-Whisper Small also provide a useful implementation-level comparison because they use the same Whisper model family while differing mainly in the inference framework. Wav2Vec2 adds a different architecture and training approach to the comparison.

This gives the benchmark three clear perspectives: general ASR capability, optimized Whisper inference, and an alternative English ASR architecture.

2.6 Model Landscape Summary

For anyone who just wants the comparison at a glance, here it is side by side:

Model	Architecture	Training Data	License	Hardware Profile	Best Fit
Whisper (base/small)	Encoder-decoder Transformer, word-by-word decoding	680k hrs, weakly supervised, 99 languages	MIT	Runs on CPU; small GPU for real time	Multilingual baseline / offline batch work
Faster-Whisper	Same Whisper weights, faster CTranslate2 runtime	Identical to Whisper (no retraining)	MIT	Low-VRAM GPU, or CPU with 8-bit mode	Whisper-level accuracy, production-ready cost
Wav2Vec2 (960h-lv60-self)	Self-supervised pretraining + one-pass CTC decode	53k hrs unlabeled + 960 hrs labeled	Apache 2.0	Lightweight, very fast single pass	Low-latency, in-domain, easy to fine-tune

3. Training Data Scale and Type

The three models were trained using different approaches and data sources, which is one of the main differences between them.

Whisper Small was trained as part of the Whisper model family using around 680,000 hours of multilingual and multitask audio collected from the web. The dataset contains speech in many languages and includes different recording conditions, speaking styles, and levels of audio quality. This broad data coverage is one reason Whisper is suitable for testing speech from less controlled environments.

Faster-Whisper Small does not have a separate training dataset. It uses the Whisper Small model weights and runs them through the CTranslate2 inference framework. Therefore, its training data is effectively the same as Whisper Small. The main difference is in how the model is executed, rather than how it was trained.

Wav2Vec2 Base 960h follows a different training approach. The specific facebook/wav2vec2-base-960h checkpoint used in this project was trained for English speech recognition using 960 hours of LibriSpeech audio. Wav2Vec2 first learns speech representations from audio and is then fine-tuned using transcribed speech. Since the selected checkpoint is closely associated with LibriSpeech, its performance on noisy and accented customer-support audio is something that needs to be measured through the benchmark rather than assumed from its clean-speech results.

Overall, Whisper and Faster-Whisper are based on the same broad, multilingual Whisper training, while Wav2Vec2 Base 960h provides a useful contrast because it is an English-focused model based on a different training and recognition approach.

3. Benchmark Methodology

3.1 How we designed the comparison

A model that tops the leaderboard on clean audiobook narration tells you very little about how it'll behave on a real support call. So rather than lean on the usual clean-speech benchmarks alone, this comparison is built around three layers of increasingly realistic — and increasingly difficult — audio:

- **Clean baseline:** LibriSpeech test-clean / test-other — the standard read-speech benchmark, used here only as a sanity-check starting point, not as the main evidence.
- **Accented speech:** Mozilla Common Voice's English test set, which draws from speakers across a wide range of first languages and regional accents.
- **Noisy, telephone-style speech:** CHiME-6 (background noise, multiple talkers, far-field microphones), CallHome (real spontaneous phone conversations), and MUSAN-based noise-injected audio — the closest public proxies we could find to an actual noisy support line.

3.2 What we measured.

Metric	Why it matters here
Word Error Rate (WER)	The core accuracy number the assignment asks for — the percentage of words the model gets wrong.
Inference time / Real-Time Factor (RTF)	Whether the model can keep pace with a live call, not just process audio eventually.
Peak memory (GPU/CPU)	Drives how many concurrent calls one server can realistically handle, and therefore cost.
Setup complexity	A plain 1–5 rating of how much engineering effort it takes to actually stand the model up.

4. Comparative Results & Analysis

4.1 Accuracy under noise and accent shift

The table below lines up word error rate for each model against the condition it's most relevant to, with the source dataset named directly so nothing is hidden behind a vague average. Faster-Whisper's numbers reflect its published Whisper large-v2-equivalent accuracy, since the whole point of the CTranslate2 rebuild is to change speed and memory without touching accuracy.

Condition (source dataset)	Faster-Whisper (large-v2)	Wav2Vec2 (large-960h)	Whisper (base/small)
Clean read speech (LibriSpeech test-clean)	~2.7%*	1.9–2.2%	1.9%
Harder clean speech (LibriSpeech test-other)	~5.2%*	3.3–5.2%	3.6%
Accented speech (Common Voice)	9.0%	29.9%	n/a†

Condition (source dataset)	Faster-Whisper (large-v2)	Wav2Vec2 (large-960h)	Whisper (base/small)
Multi-talker background noise (CHiME-6)	25.5%	65.8%	n/a†
Telephone conversational speech (CallHome)	17.6%	34.8%	n/a†
Regional US dialect speech (CORAAL)	16.2%	35.6%	n/a†
Synthetic noise injection, 9 conditions (MUSAN)	n/a†	n/a†	6.7%
Business / call-style speech (AA-WER style)	n/a†	n/a†	6.2%
Average across robustness suite	12.8%	29.3%	6.3% (Open ASR Leaderboard avg.)

4.2 Speed, throughput, and resource use

Metric	Faster-Whisper (large-v2, int8)	Wav2Vec2 (large, CTC)	Whisper (base/small)
Decoding style	Word-by-word (accelerated)	Single pass (CTC)	Frame-by-frame (transducer)
Reference throughput	13 min of audio in ~59s (V100S GPU)	Single forward pass; no published head-to-head figure for this exact suite	90 min of audio in <16s (A100 GPU, 1.1B version)
Peak GPU memory	~3.1 GB (8-bit mode)	Lower than Whisper-class models; varies by hardware	Tuned for high-throughput GPUs; TensorRT trims it further
Works on CPU alone?	Yes — documented 8-bit CPU path (~2 min for the 13-min clip)	Yes — light enough for CPU inference or fine-tuning	Possible, but not what it's optimized for
Built for live streaming?	Near-real-time with buffering	Naturally low-latency by design	Yes, natively

4.3 How hard is each one to actually deploy?

Model	Setup Effort (1 = easiest, 5 = hardest)	What that looks like in practice
Faster-Whisper	2	A pip install, plus CTranslate2 and CUDA/cuDNN for GPU use. Big community, and it's built to be a near drop-in replacement for the standard Whisper API.
Wav2Vec2	2	Standard Hugging Face install and a straightforward CTC inference loop. You'll need to add a separate model for punctuation and capitalization before it's production-ready.
Whisper (base/small)	4	Needs the full NeMo toolkit, and getting peak speed means also setting up TensorRT — most comfortable on NVIDIA hardware, and a heavier initial lift than the other two.

4.4 Putting it together for this specific use case

Weigh all four angles against what actually matters here — noisy environment, many accents, live phone calls — and Wav2Vec2 falls out of contention quickly; its accuracy simply isn't stable enough once it leaves clean, matched-domain audio. Between the two remaining candidates, Parakeet-TDT's streaming-native design and stronger published noise resilience make it the better fit for the live call path itself, while Faster-Whisper's lighter setup and broader language coverage make it the sensible partner to lean on when Parakeet can't cover something.

5. Recommendation

5.1 Run two models, not one: Parakeet-TDT primary, Faster-Whisper as backup

Rather than betting the whole system on a single model, we'd suggest a two-tier setup:

- **Tier 1 — Parakeet-TDT (0.6B):** the main engine for supported languages, chosen for its streaming-native design, the strongest published noise resilience of the three, and the best raw throughput per GPU.
- **Tier 2 — Faster-Whisper (large-v2, 8-bit):** the fallback for languages Parakeet doesn't yet cover, for overflow capacity when GPUs are busy, and as an independent second opinion during the early rollout period.

This isn't an unusual pattern — most production ASR setups end up routing by context rather than picking one model to rule everything, simply because no single open-source model currently wins on accuracy, speed, language coverage, and hardware flexibility all at once. Splitting the work this way gets the benefit of both without inheriting Wav2Vec2's fragility outside its training domain.

5.2 Ways to squeeze out more performance

- Add voice-activity detection (VAD) up front to trim silence and non-speech audio before it ever reaches the model — cheaper to run, and it cuts down on Whisper-family hallucination during dead air.

- Lean on 8-bit quantization (already reflected in the Faster-Whisper numbers above) and, where the hardware supports it, compile Parakeet with TensorRT for an extra latency cut.
- Add real telephony-style audio pre-processing — noise suppression and automatic gain control tuned to whatever noise profile actually shows up in your call logs, not a generic preset.
- Restore punctuation and capitalization automatically, either with Parakeet's punctuation-capable checkpoint or a small dedicated formatting model — raw lowercase transcripts are hard on any downstream NLU.
- Route by confidence: when a transcript segment comes back with low confidence, automatically re-run it through the Tier 2 fallback instead of sending it straight to the intent classifier.

5.3 Fine-tuning strategy

Neither model needs training from scratch — the more sensible path is targeted adaptation to your actual calls:

- **Collect real data:** build an anonymized, consent-covered corpus of in-house call recordings, and deliberately over-sample the accents and noise conditions that public benchmarks under-represent.
- **Train with noise, not just on noise:** mix in realistic background noise and simulate telephone bandwidth during fine-tuning itself, rather than only testing against noisy audio afterward.
- **Fine-tune efficiently:** use adapters or LoRA-style lightweight updates instead of retraining the whole model — cheaper to run, and far less likely to accidentally undo the noise and accent robustness the base model already earned.
- **Teach it your vocabulary:** bias the model toward product names and support jargon through a contextual biasing list or a rescoring step, rather than retraining the acoustic model just to learn a handful of proper nouns.
- **Keep checking:** spot-check a sample of live calls against human review, track real-world WER over time, and feed the hard examples back into the next fine-tuning round.

5.4 What the production pipeline looks like

In practice, audio flows through noise cleanup and voice-activity detection before it ever reaches the ASR engines, and a confidence check decides whether a segment needs a second pass through the fallback model:

Stage	Component	What it does
1	Audio comes in	Telephony/VoIP capture, resampled to 16 kHz mono, buffered per call.
2	Cleanup	Noise suppression, automatic gain control, and voice-activity detection to trim silence.
3	Main transcription	Parakeet-TDT (0.6B) streaming on a GPU pool that scales with concurrent call volume.
4	Confidence check	Low-confidence or unsupported-language segments get routed to the fallback instead of moving straight ahead.
5	Fallback transcription	Faster-Whisper (large-v2, 8-bit) on spare GPU or CPU capacity.

Stage	Component	What it does
6	Cleanup pass	Punctuation and capitalization restored, domain vocabulary corrected, PII redacted as needed.
7	Handed to the assistant	Intent detection and knowledge lookup work from the finished transcript.
8	Keeping watch	Dashboards track confidence scores, latency, and escalation rates, feeding back into fine-tuning.

6. Conclusion

The headline result here is simple, even if the underlying research took some digging: the model that wins on clean, matched-domain speech (Wav2Vec2) is not the model that should carry a real support line, because its accuracy falls apart exactly where a support line lives noise, accents, and phone-quality audio. The two Transformer- and transducer-based candidates hold up considerably better, and each brings something different to the table: broader language coverage on one side, streaming speed and noise resilience on the other. Running them together, rather than picking a single winner, gets the best of both without inheriting one's weak spot.